

Autinerary — Agent Evaluation & Data Gap Analysis

Ground truths, label distributions, and confidence formulas for all six agents

September 2026 — Internal / Confidential, not for external distribution

- [0. Housekeeping — before the agents](#)
 - [Real user numbers, corrected](#)
 - [A finding from this morning that did NOT hold up — retracted](#)
- [1. Six agents, one honesty rule](#)
- [2. Path Planning Agent](#)
- [3. Pattern Recognition Agent — the 0.2 question, and is this a “group patterns” agent?](#)
- [4. Tool Recommendation Agent](#)
- [5. Calendar Optimization Agent](#)
- [6. Reflection Analysis Agent](#)
- [7. Adaptation Agent](#)
- [8. ResourceHub has its own agent system — four more agents](#)
- [9. Recommendation Agent](#)
- [10. Pattern Agent — the agent that answers “does anything find group-level patterns?”](#)
- [11. Validation Agent](#)
- [12. Synthesis Engine](#)
- [13. What this tells us to prioritize](#)
- [14. What we need from this group](#)

0. Housekeeping — before the agents

Real user numbers, corrected

The version of this deck built this morning had a bug in how it counted users — it only ever looked at the 50 most-recently-created accounts, silently missing everyone earlier than that. Re-pulled against the full account list:

	Morning draft (wrong)	Corrected
Total accounts	50	339
Real signups	41	70
Team / QA / dev test accounts	9	19
Synthetic seed accounts	0	250

The synthetic seed cohort was never gone — it was simply outside the window the broken query could see. All 250 were created in a single batch on 2026-07-12, which is exactly what deliberate seed generation looks like.

The 70 real signups run from June 6 through today, with the bulk (~60) landing Sept 8–17. **That recent cluster is Riipen-sourced, not unexplained** — confirmed, no open question there.

A finding from this morning that did NOT hold up — retracted

This morning’s deck flagged “88 completed-milestone rows and 515 of 519 ResourceHub ratings all point at accounts that no longer exist” as an urgent data-integrity problem. **That finding was caused by the same counting bug above, not a real problem.** Once the user list is read completely, all 88 completed-milestone rows and all 519 ratings match real, currently-existing accounts. Retracted — not softened, retracted. It should not have gone out this morning, and it does not appear in this version.

The real (much less alarming) finding underneath it: those 519 ResourceHub ratings all belong to team/test/synthetic seed personas (`autism_parent@test.com` , `superautism@gmail.com` , etc.) — legitimate seed data doing what seed data is for, not a bug. **Zero ratings from real organic users yet.** That’s a genuine, calmer gap, addressed under ResourceHub below.

1. Six agents, one honesty rule

Agent	Job
Path Planning	Builds the milestone plan for a user’s goals
Pattern Recognition	Finds similar users, surfaces what worked for them
Tool Recommendation	Matches resources to a milestone’s barriers
Calendar Optimization	Places tasks into a barrier-aware weekly schedule
Reflection Analysis	Reads sentiment/patterns from journal entries
Adaptation	Adjusts the plan when reflection or completion signals change

Every agent computes confidence from something it actually measured this run — never a flat constant. Where the honest answer is “nothing to go on,” the score is 0.0.

On the numbers inside each formula below: these are engineering-chosen weights (a floor plus a scaling curve), not statistically fitted parameters. We do not yet have enough labelled outcome data to fit them properly — that itself is one of the gaps this deck is surfacing. Each slide below states plainly which numbers are “chosen so the score behaves sensibly” versus “measured directly,” so nothing is presented as more precise than it is.

2. Path Planning Agent

What “nameSource” means: every milestone the agent produces is tagged either `generated` (an LLM wrote this milestone’s name specifically for this user’s stated goals) or `template` (the LLM call for that life-area either wasn’t available or returned nothing usable, so the agent fell back to a fixed starter list every user in that life area receives). It’s a recorded fact written at generation time, not an estimate — we can always tell after the fact whether a given milestone was personalized or generic.

Ground truth: no external gold-standard path exists to compare against. The real signal is internal — how much of this specific plan was actually written for this user, per the `nameSource` flag above.

Formula: `confidence = 0.35 + 0.6 * (generated / total_milestones)`

Where 0.35 and 0.6 come from: chosen, not fitted. 0.35 is a floor — a fully templated plan is still a usable, coherent plan (just not personalized), so it shouldn’t score near zero. 0.6 is the remaining range up to a ceiling of 0.95, so a fully personalized plan reads as high-confidence without ever claiming certainty. We have not yet validated that 0.35 (versus, say, 0.3 or 0.4) is the right floor against real outcome data — flagging that honestly rather than presenting it as derived.

Real data (n=48 real paths): mean 30.9 milestones/path, range 16–144.

Gap: the generated-vs-templated split isn’t logged in queryable form yet — it’s in the payload but hasn’t been extracted into a reportable number.

Odosa’s question — should this incorporate user validation, extended from milestones to whole paths? Yes, and there’s a real gap between what exists today and what she’s describing. Tool Recommendation already has an analogous mechanism (`tool_outcomes` : a reflection-derived reward per tool, per barrier, folded back into future ranking). Path Planning’s closest equivalent (`path_planning_outcomes`) only tracks reward at the *whole-profile* level (milestone count and pacing that worked for similar profiles) — there is **no existing mechanism that rates an individual milestone against the specific barrier it was meant to help with**, and nothing that rolls milestone-level ratings up into a path-level score the way she describes (7 of 14 milestones rated useful = “half a path’s worth of value,” not just “50% complete”). This is a real, buildable extension, following the exact pattern `tool_outcomes` already established — not a new architecture.

3. Pattern Recognition Agent — the 0.2 question, and is this a “group patterns” agent?

No bare acronyms: the agent calls a Remote Procedure Call (RPC) — essentially a stored database function — named `find_similar_pattern_users`.

What this agent actually does — important scope note: this agent retrieves similar users **for one specific person, one query at a time**. It embeds *that user’s* profile and searches for their nearest

neighbours. It is **not** a population-wide, unsupervised pattern-mining system — it does not compare groups to each other or look for structure across the whole user base. (See slide 10 for the agent that actually does that, on the ResourceHub side.)

Why the confidence was 0.2: the agent requests the 10 most similar users per query — 10 is a chosen batch size, not derived from anything. With **36 real users embedded today** (corrected from an earlier, too-low estimate of 16), a query still often only finds 2 genuinely comparable people past the match-quality threshold. $2/10 = 0.2$. Worth being precise here: with the pool now known to be 36, not 16, “we just need more users” is a weaker complete explanation than it looked this morning — it may also mean the 2 real matches found were genuinely the only close matches among 36, which is a different, more specific thing to investigate (are barrier profiles just this varied, or is the match threshold too strict?).

Why we don’t use the vector database’s own similarity score: measured directly, it sits near 1.0 for every result regardless of match quality — it blends a ranking score with a success-rate bias and a personalization term. Using it would hide exactly the signal being asked about.

Formula: `confidence = min(similar_users_returned / 10, 1.0)`. The 10 and the 0.7 match-quality threshold used by the RPC are both chosen constants, not fitted or measured.

Odosa’s suggestion — should users confirm whether the returned group actually looks similar to them? Yes, and nothing like that exists today. Right now the agent’s only feedback signal is indirect, via reflections on outcomes; there’s no direct “does this group of similar users look right to you” check. That would be new instrumentation, following the same outcome-table pattern used elsewhere (`tool_outcomes`, `adaptation_outcomes`).

4. Tool Recommendation Agent

Ground truth: community ratings on the recommended resource, **plus** a learned reward from `tool_outcomes` — a reflection-derived signal per (tool, barrier) pair that’s already folded into ranking. This wasn’t fully stated this morning: the formula isn’t rating-only, it already incorporates outcome learning.

Formula: `evidence = 0.6 + 0.4*(rated/total)` then `confidence = mean_relevance * evidence`. No catalogue match $\rightarrow 0.0$.

Where 0.6 and 0.4 come from: chosen. 0.6 is a floor so relevance still counts for something even with zero reviews; 0.4 is the remaining weight given to how much of the recommended set carries real community evidence. Not fitted against outcome data yet.

Real data: 132 resources, 95 approved / 37 pending. Thinnest categories: Employment (5), Housing (4) — also two of the highest-stakes categories for adult independence.

Gap: review-coverage across the 95 approved resources hasn’t been queried yet — it’s the direct ceiling on this agent’s confidence.

Odosa's question — does this (or any agent) consider people: mentors, role models, friends, rivals?

No. The `connections` table exists in the schema (role_model / mentor / friend / rival relationships are already modelled and stored) but **no agent, in either system, currently reads it**. This is a real, clean gap, not a design decision we made on purpose — social connections are captured but never used in a recommendation. Whether to build this as a genuinely combined “Tool & People Recommendation Agent,” or keep them separate and just wire the existing connections data into this agent’s scoring, is a real design choice worth a deliberate decision rather than defaulting into it.

5. Calendar Optimization Agent

Ground truth, plainly: the agent is not graded on “did it get through everyone’s whole to-do list.” It’s graded on “did it produce a sensible week” — every day has a reasonable amount to do, nothing is crammed, nothing is empty when it didn’t need to be. A real run once placed only 30 of a person’s 80 backlogged tasks into one week’s schedule, and that was the *correct* outcome, not a failure: 80 tasks across 16 milestones is genuinely several weeks of real work, and a schedule that forced all of it into 7 days would be actively bad for the user, even though it would look more “complete” by a naive measure.

Formula: `fill = days_with_work/total_days`, then `confidence = fill * (1.0 if preferred_buckets else 0.75)`.

Where 1.0 and 0.75 come from: chosen. A user who’s told us their preferred time-of-day buckets gets full credit (1.0) for a well-formed week; without that preference data the agent is working with less to go on, so the same well-formed week is capped at 0.75 confidence rather than claiming the same certainty. Not fitted against real satisfaction data.

Gap: real per-user weekly fill rate hasn’t been pulled across the actual user base yet.

6. Reflection Analysis Agent

Ground truth: length and pattern-density of what the user actually wrote. A one-word entry cannot score as high as a paragraph — a hard floor, not an inference the agent makes anyway.

Formula: `depth = min(words/60, 1.0)`, `signal = min(patterns/3, 1.0)`, `confidence = 0.3 + 0.5*depth + 0.2*signal` (0.0 if empty).

Where these numbers come from: chosen. 60 words is a judgment call for “this reads like a real reflection, not a one-liner” — untested against real data on whether that’s the right cutoff. 3 detected patterns is treated as “plenty of signal” for the same reason. The 0.3 / 0.5 / 0.2 split weights depth of writing over pattern count, on the reasoning that length is a more reliable signal than our current pattern-detection quality — again, a judgment call, not a fitted weighting.

Gap: entry-length distribution across real reflections hasn’t been pulled for this pass.

7. Adaptation Agent

Ground truth today: whether the agent found something to actually adapt. Zero adaptations fired is a real, different answer from a low-confidence guess.

Formula: `confidence = min(0.4 + 0.15 * adaptations_fired, 1.0)` (0.0 if none fired). 0.4 and 0.15 are chosen constants — a floor once *anything* was adapted, plus 0.15 credit per adaptation, capping out after roughly 4.

Correction from this morning: we said no adaptation-events log existed at all. That was wrong. A table (`adaptation_outcomes`) already exists, designed to record exactly the kind of signal Odosa is asking for — which rule fired, and what the user's *next* reflection reward looked like afterward, as an implicit “did this seem to help.” **The table exists and has the right shape. It currently has zero rows — the mechanism was built but never wired up to actually write to it.** That's a smaller, more precise gap than “we need to design this from scratch.”

Odosa's critique — this is weak, because firing more adaptations isn't evidence they worked.

Correct, and sharper than our framing. The current formula literally rewards quantity of changes, not quality. Her proposed fix — suggest an adaptation, get an explicit yes/no, then re-ask after a set period (a week) whether it actually helped, and score both moments — is a real improvement over the current implicit, reflection-only signal, and it maps directly onto the unused `adaptation_outcomes` table: it already has a slot for “reward from what happened next,” it just needs (1) writes turned on, and (2) an explicit prompt-based signal added alongside the passive reflection-derived one.

8. ResourceHub has its own agent system — four more agents

Everything above is Autinerary's path-planning orchestrator (`backend/core/agents/`). ResourceHub (`servicehub-mvp/`) runs a **separate** set of four agents, on the same Supabase project but with its own LLM client and its own orchestrator. Presenting six agents without these four would be presenting half the product.

Agent	Job
Recommendation Agent	Matches people to resources (collaborative filtering + LLM explanation)
Pattern Agent	Discovers patterns across the whole user population, unprompted
Validation Agent	Approves / rejects / flags submitted content and ratings
Synthesis Engine	Combines the three above into one ranked, explained result

Finding common to Recommendation and Validation: ResourceHub’s 519 ratings all belong to team/test/synthetic seed personas — 0 from a real organic user yet. Recommendation Agent quality and Validation Agent trust scores both ultimately wait on this same gap: real people rating real resources.

9. Recommendation Agent

Ground truth: collaborative filtering against similar users’ resource ratings, with an LLM explanation layer and cross-session memory. Cold-start fallback ranks approved resources directly when vector similarity finds no neighbours, so a new user is never shown “no results.”

Formula: $\text{confidence} = \min(\text{similarUsers}/50, 1.0) * 50 + (\text{avgScore}/100) * 50$ — half from how many comparable users were found, half from mean recommendation quality.

Where 50/50 and the /50 denominator come from: chosen. The split gives equal weight to “did we find enough comparable people” and “how good does the match look,” on the reasoning that either one alone can mislead — many weak matches or one perfect match with no corroboration are both worth discounting. The specific denominator (50 similar users = full credit) is not derived from data.

Real data: 95 approved resources, 72 (76%) have ≥ 1 rating — but per slide 8, those ratings are all seed/test data, so the “similar users” half of this formula has no real signal to draw on yet.

Same people/mentors question as Tool Recommendation: no, this agent also does not read the `connections` table.

10. Pattern Agent — the agent that answers “does anything find group-level patterns?”

This is the direct answer to the question raised for today: **is there an agent that finds overarching patterns between groups and within groups, unsupervised** — as opposed to patterns for one individual at a time?

Short answer: partially, and only on the ResourceHub side. Autinerary’s Pattern Recognition Agent (slide 3) is strictly per-user retrieval — it never looks at the population as a whole. This ResourceHub Pattern Agent is the one that actually mines the whole population, unsupervised, without a predefined target — closer to what “pattern recognition” sounds like it should mean.

What it actually finds, precisely, so we don’t overclaim: - **Within-group co-occurrence** — which conditions cluster together in the same people across the whole population (e.g. “43 users have Autism + ADHD together”). This is aggregate, unsupervised, population-level. - **Resource affinity** — population-level collaborative filtering (“people who rated X highly also rated Y highly”). - **Unexpected preferences** — which barrier types over-index on liking a given resource category, aggregated across everyone who rated it.

What it does NOT do, and why that’s a real architectural point, not an oversight: it does not run explicit **between-named-group** comparisons (e.g. Autism community vs. a racial/ethnic community as two labelled cohorts), and it **cannot currently compare diagnosis subtypes** (e.g. Autism Level 1 vs. Level 3) — because that subtype-level detail is deliberately stripped out before any data reaches an agent (the same privacy boundary covered in the diagnostic-profile design: agents get “what support helps,” never the underlying label or its severity level). Enabling subtype-level group comparison would mean deliberately relaxing that boundary for this one use case — a considered decision, not a small code change, and one that should be made explicitly rather than as a side effect of adding this feature.

Real data: only 48 real users have any barrier data, and only 25 of those have 2+ distinct barrier types — below the default 5-user minimum support needed for most combinations to surface. Same root cause as Autinerary’s Pattern Recognition Agent, same fix: more real users.

11. Validation Agent

Ground truth: four independent checks (Content Quality, Spam Detection, User Trust, Behavioral Patterns) combine into approve / reject / flag-for-review. `confidence = mean of the four check scores`.

Decision rule: approve if `trust > 70` & `quality > 80` & `spam < 30`; reject if `trust < 30` OR `quality < 40` OR `spam ≥ 70`; otherwise flag for human review. The specific cutoffs (70/80/30, 30/40/70) are chosen thresholds, not fitted — the deliberately wide middle band that defaults to human review is the one part of this that’s a considered design choice rather than an arbitrary number: it’s there so uncertain cases go to a person, not so the system looks more automated than it is.

Trust score: starts at 50 (chosen baseline), +up to 20 for account age (roughly 2 points/day, capped), +up to 20 for prior contributions (2 points each, capped), +up to 10 for helpful votes received, –up to 30 for violations (15 points each, capped), + a consistency bonus. All bounds are chosen caps to keep any single factor from dominating, not fitted weights.

Real data: `moderation_queue` has **0 rows**. The approve/reject/flag logic exists and runs, but nothing has been logged to the table that would let us evaluate it against real decisions yet.

12. Synthesis Engine

Combines Recommendation Agent + Pattern Agent + Validation Agent output into one ranked, explained result — filters out anything Validation rejected, generates human-readable explanations, and picks a synthesis strategy that's bandit-refined against learned reward when enough signal exists.

Ground truth: none independently — its correctness is entirely a function of the three agents it combines. There's no separate metric to evaluate Synthesis Engine on its own; if Recommendation, Pattern, and Validation are each sound, Synthesis inherits that. If any of the three's real-data gaps above aren't closed, Synthesis can't be evaluated meaningfully either.

13. What this tells us to prioritize

Agent	Status	What would move it
Pattern Recognition (Autinerary)	Explained; pool bigger than we thought (36) but still small	More real users with embeddings
Tool Recommendation	Formula solid, one query missing	Review-coverage across 95 approved resources
Path Planning	Formula solid; milestone-level validation doesn't exist yet	Extract nameSource split; design milestone/path rating loop
Calendar Optimization	Formula solid, one query missing	Real per-user weekly fill rate
Reflection Analysis	Formula solid, one query missing	Entry-length distribution
Adaptation	Infra exists (adaptation_outcomes), unused	Wire up writes; add explicit yes/no + delayed re-ask
Recommendation (ResourceHub)	Formula solid, ratings are all seed data	Get real users rating real resources
Pattern (ResourceHub)	Explained, expected	More users with ≥ 2 barrier types (25 today)
Validation (ResourceHub)	No decision log yet	Start logging to moderation_queue

Most of these are queries against data we already have or infrastructure that already exists and just needs wiring up — not new experiments. The two Pattern agents share the same root fix (more real users), already our #1 roadmap item.

14. What we need from this group

October beta timing: hold ResourceHub's public launch to waitlist + feature demo only, or move fast on data-compliance work, before going fully public for Autism Awareness Month. This is sharper for ResourceHub specifically — a community rating system that scores things like “autism-friendliness” sits closer to the medical-device-classification line Malikeh flagged than Autinerary's planning agents do.

Everything in this deck is reproducible: `backend/scripts/vector_evaluation_pull.py` and `backend/scripts/vector_evaluation_pull_servicehub.py`, plus the agent source files in `backend/core/agents/` and `servicehub-mvp/lib/agents/`.